

Лабораторная работа №7: Global State, API и CRUD

1. Требования к Авторизации (AuthContext)

Доступ к каталогу должен быть ограничен для неавторизованного пользователя.

- Создать `AuthContext`, хранящий объект `user` и флаг `isAuthenticated`.
- При загрузке приложения проверять наличие данных пользователя в `localStorage`.
- Реализовать компонент-обертку (или логику в `App.tsx`), которая:
 - Если пользователь не авторизован — показывает форму регистрации (из Лабы №6).
 - Если авторизован — рендерит `CatalogPage`.
- В шапке каталога должна быть кнопка «Выйти», очищающая стейт и `localStorage`.

2. Использование API (DummyJSON)

Все операции с данными должны имитировать работу с реальным бэкендом.

- **Базовый URL:** `https://dummyjson.com/products`.
- При первом рендере каталога отправлять `GET` запрос.

3. Реализация CRUD операций

Управление товарами должно происходить через `useReducer` в `ProductContext`.

Create	Кнопка «Добавить» открывает форму. После заполнения — <code>POST</code> запрос. При успехе (200 OK) товар добавляется в начало локального списка.
Read	Отображение списка товаров в виде сетки (<code>Grid</code>).
Update	Кнопка «Изменить» предзаполняет форму данными выбранного товара. После правки — <code>PUT</code> запрос и обновление элемента в массиве через <code>.map()</code> .
Delete	Кнопка «Удалить» отправляет <code>DELETE</code> запрос. При успехе — удаление из локального стейта через <code>.filter()</code> .

4. Валидация через Zod

Форма не должна отправлять данные на сервер, если они не валидны.

- **Схема (`ProductSchema`):**
 - `title`: минимум 3 символа.
 - `price`: число, строго больше 0.
 - `id`: число (`number`).
- **Обработка ошибок:**
 - Использовать метод `.safeParse()`.
 - В случае ошибки **запрещено** допускать "падение" приложения.
 - Ошибки должны выводиться пользователю (через `alert` или текстовые поля) в понятном виде (например, "title: Название слишком короткое").
 - Использовать метод `.flatten()` для безопасного извлечения текстов ошибок.

5. Интерфейс

- Разделить код на `ProductContext`, `Catalog` (основной экран) и мелкие UI-элементы.
- Запрещено применять `any`. Использовать `z.infer<typeof ProductSchema>` для типизации товаров.

Контрольные вопросы

1. В чем концептуальное отличие `useReducer` от `useState`? В каких случаях предпочтительнее использовать первый?
2. Какую роль выполняет функция `dispatch`? Почему мы передаем в нее объект с полем `type`?
3. Почему запрос за списком товаров (GET) необходимо оборачивать в хук `useEffect`? Что произойдет, если вызвать `fetch` прямо в теле компонента?
4. Объясните логику обновления стейта при удалении (DELETE) товара. Почему мы сначала ждем ответа от сервера, а затем вызываем `dispatch` с методом `.filter()`?
5. В чем разница между POST и PUT запросами в контексте добавления и изменения товара?
6. Что возвращает метод `ProductSchema.safeParse(data)` в случае успеха и в случае провала валидации?
7. Что делает конструкция `z.infer<typeof ProductSchema>` и какую проблему двойного написания кода она решает?
8. Как приложение «понимает», что пользователь авторизован, если он закрыл вкладку и открыл ее заново через час? Опишите механизм восстановления сессии.